

8. Részletes tervek

62 - banducole

Konzulens:

Szabó András

Csapattagok

Tullner Levente Botond	APJGDR	1234tl12@gmail.com
Dekkert Bence	S2V3EZ	deki.bence@gmail.com
Kiss Ákos Attila	FTS6X7	kissakos2005@gmail.com
Maruzsán Borisz	UJO4ZE	maruzsanborisz05@gmail.com
Stumpf Ádám György	NFTG9H	stumpfadam05@gmail.com

2026.04.19

8. Részletes tervek

8.1 Osztályok és metódusok tervei.

8.1.1 Game

- **Felelősség**

A játékmenet legfelsőbb szintű koordinálása. Tárolja a várost, a játékosokat és az aktuális kör számát. Felelőssége a játék elindítása, a körök léptetése és a győzelmi feltételek ellenőrzése.

- **Ősosztályok**

-

- **Interfészek**

-

- **Attribútumok**

- - city: City: A szimulált várostároló és végrehajtó objektum.
- - players: List<Player>: A játékban résztvevő összes játékos (takarítók és buszvezetők) listája.
- - roundNumber: int: Az aktuális kör sorszáma.

- **Metódusok**

- + Start(playerCount: int, playerTypes: List<PlayerType>): void: Inicializálja a várost és létrehozza a megadott számú és típusú játékost.
- + NextRound(): void: Végrehajtja a következő kört: meghívja minden játékos TakeTurn() metódusát, majd a várost lépteti.
 - o Algoritmus:
 CIKLUS jatekos ELEME players:
 jatekos.TakeTurn()
 city.ExecuteStep()
 roundNumber++
- + IsGameOver(): boolean: Megállapítja, hogy a játéknak vége van-e.
 - o Algoritmus:
 CIKLUS csomopont ELEME city.nodes:
 CIKLUS ut ELEME csomopont.GetConnectedRoads():
 CIKLUS sav ELEME ut.GetLanes():
 CIKLUS jarmu ELEME sav.vehicles:
 HA jarmu.IsBlocked() == hamis:
 VISSZA hamis
 VISSZA igaz

8.1.2 City

- **Felelősség**

A város közlekedési hálózatának központi nyilvántartója. Ez az osztály tartja számon a játékban szereplő összes csomópontot (kereszteződéseket, végállomásokat, stb.). Feladata a játék menetének irányítása, az aktív elemek (járművek, hókotrók) lépéseinek összehangolása, és az időjárási események (például a hóesés) kezelése a teljes úthálózaton.

- **Ősosztályok**

-

- **Interfészek**

-

- **Attribútumok**

- - nodes: List<Node>: A városban található összes csomópont (kereszteződés, végállomás stb.) listája.

- **Metódusok**

- + ExecuteStep(): void: Végigiterál a város összes aktív elemén, amit nem játékos irányít, és meghívja azok Step() metódusát a szimuláció előrehaladásához.
- + ApplySnowfall(amount: int): void: Az egész várost érintő hóesést szimulál. Bejárja a csomópontokat, és minden kapcsolódó útra egyszer továbbítja a hóesést.

- Algoritmus:

HALMAZ marLatogatottUtak

CIKLUS node ELEME nodes:

CIKLUS ut ELEME node.GetConnectedRoads():

HA ut NINCS BENNE marLatogatottUtak:

ut.ApplySnow(amount)

marLatogatottUtak.ADD(ut)

8.1.3 PathFinder

- **Felelősség**

Az útvonalkeresési logika implementálásáért felelős. Meghatározza a legrövidebb utat két csomópont között, valamint kiszámítja a szomszédos sávokat és a járható útvonalakat.

- **Ősosztályok**

-

- **Interfészek**

-

- **Attribútumok**

- - city: City: Referencia a városmodellre az úthálózat eléréséhez.

- **Metódusok**

- + GetShortestPath(from: Node, to: Node): List<Lane>: Szélességi keresés (BFS) segítségével kiszámítja a legrövidebb útvonalat két csomópont között, és visszaadja a sávok listáját.
 - Algoritmus:
 QUEUE s; s.PUSH(from)
 Minden node-hoz TAVOLSAG = végtelen; ELOZO = null; from.TAVOLSAG = 0;
 AMÍG s NEM ÜRES:
 u = s.POP()
 HA u == to: Útvonal visszafejtése sávok listájaként az ELOZO mutatók alapján, VISSZA lista
 CIKLUS ut ELEME u.GetConnectedRoads():
 v = GetOtherEnd(ut, u)
 HA v.TAVOLSAG == végtelen:
 v.TAVOLSAG = u.TAVOLSAG + 1; v.ELOZO = u;
 s.PUSH(v);
- + GetNeighborLane(road: Road, lane: Lane, direction: Direction): Lane: Visszaadja a megadott sáv melletti sávot az adott irányban (LEFT/RIGHT) az úton belül.
 - Algoritmus:
 savok = road.GetLanes()
 index = savok.IndexOf(lane) // Megkeressük hányadik sávban vagyunk
 HA direction == LEFT ÉS index > 0:
 VISSZA savok[index - 1]
 KÜLÖNBEN HA direction == RIGHT ÉS index < savok.Size() - 1:
 VISSZA savok[index + 1]
 VISSZA null
- + GetOtherEnd(road: Road, node: Node): Node: Visszaadja az út túlsó végén lévő csomópontot a megadott kezdőponthoz képest.
- + GetPassableLanes(road: Road): List<Lane>: Visszaadja az úton lévő összes olyan sávot, amelyen jelenleg lehetséges a közlekedés.

8.1.4 Road

- **Felelősség**

Két csomópont közötti útszakaszt modellez. Tárolja és kezeli a hozzá tartozó sávok listáját, valamint továbbítja a környezeti hatásokat (hóesés) a sávoknak.

- **Ősosztályok**

-

- **Interfészek**

-

- **Attribútumok**

- - from: Node: Az út kiindulási csomópontja.
- - to: Node: Az út cél csomópontja.
- - lanes: List<Lane>: Az úthoz tartozó forgalmi sávok listája.

- **Metódusok**

- + GetLanes(): List<Lane>: Visszaadja az út összes sávját.
- + ApplySnow(amount: int): void: Meghívja minden sávján az AddSnow(amount) metódust.
 - Algoritmus:
 CIKLUS sav ELEME lanes:
 sav.AddSnow(amount)

8.1.5 Node (absztrakt)

- **Felelősség**

Absztrakt alaposztály, amely az úthálózat csomópontjait reprezentálja. Feladata a csomópontokhoz csatlakozó utak nyilvántartása és azok elérésének biztosítása az útvonalkereséshez.

- **Ősosztályok**

-

- **Interfészek**

-

- **Attribútumok**

- - connectedRoads: List<Road>: A csomópontokhoz fizikailag kapcsolódó utak listája.

- **Metódusok**

- + GetConnectedRoads(): List<Road>: Visszaadja a csomópontokhoz csatlakozó összes utat.
- + AddRoad(road: Road): void: Új utat ad hozzá a csomópont belső listájához.

8.1.6 Intersection

- **Felelősség**

Egy általános útkereszteződést modellez a városban. A Node osztály konkrét megvalósításaként szolgál az úthálózat építéséhez.

- **Ősosztályok**

Node

- **Interfészek**

-

- **Attribútumok**

-

- **Metódusok**

- + AddRoad(road: Road): void: Meghívja az ősosztály AddRoad metódusát az út rögzítéséhez.

8.1.7 Terminal

- **Felelősség**

Buszvégállomást modellez, amely a buszok útvonalának egyik kezdő- vagy végpontja. Felelőssége értesíteni a buszt a megérkezésről és kezelni a forduló teljesítését.

- **Ősosztályok**

Node

- **Interfészek**

-

- **Attribútumok**

-

- **Metódusok**

- + NotifyArrival(bus: Bus): void: Értesítést kap egy busz megérkezéséről, és meghívja a busz ArrivedAtTerminal() metódusát.
- + AddRoad(road: Road): void: Meghívja az őosztály AddRoad metódusát az út rögzítéséhez.

8.1.8 Home

- **Felelősség**

Az autók lakóhelyi célállomását reprezentáló csomópont. Az autók navigációja során az útvonal egyik lehetséges végpontjaként szolgál.

- **Ősosztályok**

Node

- **Interfészek**

-

- **Attribútumok**

-

- **Metódusok**

- + AddRoad(road: Road): void: Meghívja az őosztály AddRoad metódusát az út rögzítéséhez.

8.1.9 Workplace

- **Felelősség**

Az autók munkahelyi célállomását reprezentáló csomópont. Az autók navigációja során az útvonal egyik lehetséges végpontjaként szolgál.

- **Össztályok**

Node

- **Interfészek**

-

- **Attribútumok**

-

- **Metódusok**

- + AddRoad(road: Road): void: Meghívja az osztály AddRoad metódusát az út rögzítéséhez.

8.1.10 Lane

- **Felelősség**

Egy forgalmi sávot modellez az úton belül. Nyilvántartja a rajta tartózkodó járműveket, a sáv aktuális fizikai állapotát (LaneState), kezeli a sózást, a zúzalékot és a jégképződéshez szükséges áthaladás számlálót.

- **Össztályok**

-

- **Interfészek**

-

- **Attribútumok**

- - state: LaneState: A sáv aktuális állapotát leíró objektum.
- - vehicles: List<Vehicle>: A sávon éppen tartózkodó járművek listája.
- - snowPlow: SnowPlow: A sávon lévő hókotró (maximum egy).
- - saltCountdown: int: A só hatásából hátralévő körök száma.
- - passageCount: int: Az utolsó jégképződés óta áthaladt járművek száma.
- - rocky: boolean: Jelzi, hogy a sáv le van-e szórva zúzalékkal (kavicsal).

- **Metódusok**

- + Accept(v: Vehicle): void: Fogadja a sávra belépő járművet. Ha a sáv járhatatlan, a járművet blokkolt állapotba helyezi.
 - Algoritmus:
 HA IsPassable() == igaz:
 vehicles.ADD(v)

RegisterPassage(v)

- + Accept(s: SnowPlow): void: Fogadja a sávra belépő hókotrót (a hókotró mindig be tud lépni).
- + AddSnow(amount: int): void: Havat ad a sávhoz, de csak akkor, ha a só védőhatása már elmúlt.
 - Algoritmus:
 HA saltCountdown > 0:
 saltCountdown--
 KÜLÖNBEN:
 state.OnSnowAdded(THIS, amount)
- + ApplyRock(): void: Zúzalékot szór a sávra, a rocky attribútumot igazra állítja.
- + ApplySalt(): void: Sót szór a sávra, elindítja a visszaszámlálót, és delegálja a hatást az állapotnak.
- + FormIce(): void: Kiváltja a jégképződést a sávon, delegálva a hívást az aktuális állapotnak.
- + IsPassable(): boolean: Visszaadja, hogy a sáv járható-e járművek számára.
 - Algoritmus:
 CIKLUS jarmu ELEME vehicles:
 HA jarmu.IsBlocked() == igaz:
 VISSZA hamis
 VISSZA igaz
- + IsSlippery(): boolean: Visszaadja, hogy a sáv csúszik-e. A zúzalék felülbírálja a jég csúszósságát.
 - Algoritmus:
 HA rocky == igaz:
 VISSZA hamis
 VISSZA state.IsSlippery()
- + RegisterPassage(v: Vehicle): void: Növeli az áthaladás számlálót, és ha eléri a küszöböt, jeget képez.
 - Algoritmus:
 passageCount++
 HA passageCount >= KÜSZÖB:
 FormIce()
 passageCount = 0
- + CleanWith(plow: SnowPlow): void: Meghívja a hókotróra felszerelt fej tisztító algoritmusát a sávon.
- + SetState(state: LaneState): void: Megváltoztatja a sáv aktuális fizikai állapotát.
- + GetRoad(): Road: Visszaadja az utat, amelyhez a sáv tartozik.

8.1.11 LaneState (interfész)

- **Felelősség**

Interfész a sávok állapotainak (State Pattern) egységes kezeléséhez. Definiálja az állapotfüggő viselkedéseket a havazásra, szózásra és áthaladásra vonatkozóan .

- **Ősosztályok**

-

- **Interfészek**

-

- **Attribútumok**

-

- **Metódusok**

- + OnSnowAdded(lane: Lane, amount: int): void: Kezeli a hó hozzáadását az adott állapotban.
- + OnSaltApplied(lane: Lane): void: Kezeli a só alkalmazását az adott állapotban.
- + OnIceFormed(lane: Lane): void: Kezeli a jégképződés hatását az adott állapotban.
- + IsPassable(): boolean: Megadja, hogy az adott állapot járható-e járművek számára.
- + IsSlippery(): boolean: Megadja, hogy az adott állapot csúszós-e alapértelmezetten.
- + CanBePushed(): boolean: Megadja, hogy az állapot tartalma eltávolítható-e mechanikus fejjel.

8.1.12 ClearState

- **Felelősség**

A tiszta sáv állapotát reprezentálja. Sem hó, sem jég nincs a sávon. Átjárható és nem csúszós.

- **Ősosztályok**

-

- **Interfészek**

LaneState

- **Attribútumok**

-

- **Metódusok**

- + OnSnowAdded(lane: Lane, amount: int): void: Átváltja a sávot ThinSnowState állapotba.
 - Algoritmus:

```
lane.SetState(NEW ThinSnowState(amount))
```

- + OnSaltApplied(lane: Lane): void: Nincs állapotváltoztató hatása.
- + OnIceFormed(lane: Lane): void: Nincs hatása.
- + IsPassable(): boolean: VISSZA igaz.
- + IsSlippery(): boolean: VISSZA hamis.
- + CanBePushed(): boolean: VISSZA hamis.

8.1.13 ThinSnowState

- **Felelősség**

Vékony hóréteggel borított sávot reprezentál. Járható, nem csúszós. További havazás esetén vastag hóba, sok áthaladás esetén jeges állapotba vált.

- **Össztályok**

-

- **Interfészek**

LaneState

- **Attribútumok**

- - snowThickness: int: A hóréteg vastagsága.

- **Metódusok**

- + OnSnowAdded(lane: Lane, amount: int): void: Növeli a hóvastagságot. Ha eléri a határt, ThickSnowState-re vált.
 - Algoritmus:

```
snowThickness += amount
HA snowThickness >= VASTAG_HO_KUSZOB:
    lane.SetState(NEW ThickSnowState(snowThickness))
```
- + OnSaltApplied(lane: Lane): void: A só elolvasztja a vékony havat.
- + OnIceFormed(lane: Lane): void: A letaposott hó megfagy.
- + IsPassable(): boolean: VISSZA igaz.
- + IsSlippery(): boolean: VISSZA hamis.
- + CanBePushed(): boolean: VISSZA igaz.

8.1.14 ThickSnowState

- **Felelősség**

A vastag hóval borított sáv állapotot reprezentálja. Járművek számára elakadást okoz (nem járható), de hókotrók számára átjárható .

- **Ősosztályok**

-

- **Interfészek**

LaneState

- **Attribútumok**

- - snowThickness: int: A hóréteg vastagsága.

- **Metódusok**

- + OnSnowAdded(lane: Lane, amount: int): void: Növeli a hóvastagságot, állapot nem változik.
- + OnSaltApplied(lane: Lane): void: A só hatására a hó olvadni kezd (átválthat ThinSnowState-be).
 - Algoritmus:
snowThickness -= OLVADAS_MERTEK
HA snowThickness < VASTAG_HO_KUSZOB:
lane.SetState(NEW ThinSnowState(snowThickness))
- + OnIceFormed(lane: Lane): void: Nincs hatása.
- + IsPassable(): boolean: VISSZA hamis.
- + IsSlippery(): boolean: VISSZA hamis.
- + CanBePushed(): boolean: VISSZA igaz.

8.1.15 IcyState

- **Felelősség**

A jeges sáv állapotát reprezentálja. Vékony havas sávon sok jármű áthaladása után alakul ki. A sáv járható, de csúszós, a járművek megcsúszhatnak rajta .

- **Ősosztályok**

-

- **Interfészek**

LaneState

- **Attribútumok**

- - iceThickness: int: A jégréteg vastagsága.

- **Metódusok**

- + OnSnowAdded(lane: Lane, amount: int): void: Hó ráhullása vastagítja a jégréteget, de az állapot nem változik.
- + OnSaltApplied(lane: Lane): void: A só felolvasztja a jeget, a sáv tiszta lesz.
- + OnIceFormed(lane: Lane): void: Nincs hatása.
- + IsPassable(): boolean: VISSZA igaz.
- + IsSlippery(): boolean: VISSZA igaz.
- + CanBePushed(): boolean: VISSZA hamis.

8.1.16 BrokenIceState

- **Felelősség**

A feltört jég állapotot reprezentálja. Akkor kerül egy sáv ebbe az állapotba, ha jégtörő fejjel dolgoztak rajta. Járható, de csúszós, és letolható.

- **Ősosztályok**

-

- **Interfészek**

LaneState

- **Attribútumok**

- - iceThickness: int: A feltört jég vastagsága.

- **Metódusok**

- + OnSnowAdded(lane: Lane, amount: int): void: Növeli a vastagságot.
- + OnSaltApplied(lane: Lane): void: A só felolvasztja a törmeléket, a sáv tiszta lesz.
- + OnIceFormed(lane: Lane): void: Nincs hatása. + IsPassable(): boolean: VISSZA igaz.
- + IsSlippery(): boolean: VISSZA igaz.
- + CanBePushed(): boolean: VISSZA igaz.

8.1.17 CleanerHead (absztrakt)

- **Felelősség:**

Absztrakt őszosztály a különböző takarítófejek számára. Definiálja az általános tisztítási interfészt, amelyet a leszármazottak valósítanak meg.

- **Őszosztályok**

-

- **Interfészek**

-

- **Attribútumok**

-price: int: A fej megvásárlási ára

- **Metódusok**

- + Clean(lane: Lane): void (Absztrakt): A megadott sávon elvégzi a takarítást.
- + IsOperational(): boolean (Absztrakt): Visszaadja, hogy a fej működő képes e.
- + GetPrice(): int: Visszaadja a fej árát.

8.1.18 SweepHead

- **Felelősség:**

A söprő fejért felelős osztály. Feladata a havat és a zúzalékot az aktuális sávval szomszédos sávba áttolni

- **Őszosztályok:**

CleanerHead

- **Interfészek:**

-

- **Attribútumok:**

-

- **Metódusok:**

- + Clean(lane: Lane): void: Áttolja a havat vagy zúzalékot a szomszédos sávba, ha az állapot megengedi.

- Algoritmus:

allapot = lane.GetState()

HA allapot.CanBePushed() == igaz:

ut = lane.GetRoad()

szomszedSav = pathFinder.GetNeighborLane(ut, lane, RIGHT)

HA szomszedSav != null:

szomszedSav.AddSnow(amount)

lane.rocky = hamis

lane.SetState(NEW ClearState())

- + IsOperational(): boolean: VISSZA igaz

8.1.19 ThrowHead

- Felelősség:

A hóhányó fejét felelős osztály. A havat és zúzalékot az út mellé szórja ki

- Ősosztályok:

CleanerHead

- Interfészek:

-

- Attribútumok:

-

- Metódusok:

- + Clean(lane: Lane): void: Eltávolítja a havat és a zúzalékot a sáv állapotától függően.

- Algoritmus:

```

allapot = lane.GetState()
HA allapot.CanBePushed() == igaz:
    lane.rocky = hamis
    lane.SetState(NEW ClearState())

```

- IsOperational(): boolean: VISSZA igaz

8.1.20 IceBreakerHead

- Felelősség:

A jégtörő fejét felelős osztály. Feladata a jégréteg feltörése, ami után a jég már letolhatóvá válik.

- Ősosztályok:

CleanerHead

- Interfészek:

-

- Attribútumok:

-

- Metódusok:

- + Clean(lane: Lane): void: Ha a sáv jeges, feltöri azt és BrokenIceState-be vált.

- Algoritmus:

```

allapot = lane.GetState()
HA allapot TÍPUSA IcyState:
    lane.SetState(NEW BrokenIceState())

```

- + IsOperational(): boolean: VISSZA: igaz

8.1.21 SaltHead

- Felelősség:

Sószóró fej, amely segít felolvasztani a havat és a jeget.

- Ősosztályok:

CleanerHead

- Interfészek:

-

- Attribútumok:

- salt: Salt: Referencia a használt só típusú üzemanyag objektumra.
- saltamount: int: A tartályban lévő só aktuális mennyisége.

- Metódusok:

- + Clean(lane: Lane): void: Sózást alkalmaz a sávon, ha van elég készlet.
 - Algoritmus:

```

HA IsOperational() == igaz:
    lane.ApplySalt()
    saltamount = saltamount - 1

```

- + IsOperational(): boolean: Ellenőrzi, hogy van-e só a tartályban.
 - Algoritmus:

```

HA saltamount > 0:
    VISSZA igaz
    VISSZA hamis

```

- + Refuel(amount: int): void: Növeli a só mennyiségét a megadott értékkel.

8.1.22 DragonHead

- Felelősség:

Kerozinnal működő olvasztófej, amely azonnal eltünteti a jeget.

- Ősosztályok:

CleanerHead

- Interfészek:
 -
- Attribútumok:
 - kerosene: Kerosene: Referencia a kerozin típusú üzemanyag objektumra.
 - keroseneamount: int: A tartályban lévő kerozin aktuális mennyisége.
- Metódusok:
 - + Clean(lane: Lane): void: Mindent eltávolít a sávról kerozin felhasználásával.
 - Algoritmus

HA IsOperational() == igaz:

lane.SetState(NEW ClearState())

keroseneamount = keroseneamount - 1

- + IsOperational(): boolean: Ellenőrzi a kerozinszintet.
 - Algoritmus:

HA keroseneamount > 0:

VISSZA igaz

VISSZA hamis

- + Refuel(amount: int): void: Növeli a kerozin mennyiségét a megadott értékkel.

8.1.23 RockHead

- Felelősség:

Zúzottkő-szóró fej, amely megszünteti a jég csúszósságát.
- Össztályok:

CleanerHead
- Interfészek:
 -
- Attribútumok:
 - rocklimit: int: A zúzaléktartály maximális kapacitása.
 - rockamount: int: A rendelkezésre álló zúzalék aktuális mennyisége.
 - rock: Rock: Referencia a zúzottkő objektumra.
- Metódusok:
 - + Clean(lane: Lane): void: Zúzalékot szór a sávra, meggátolva a megcsúszást.
 - Algoritmus:

```

HA IsOperational() == igaz:
    lane.ApplyRock()
    rockamount = rockamount - 1

```

- IsOperational(): boolean: Ellenőrzi a zúzalékkészletet.
 - Algoritmus:

```

HA rockamount > 0:
    VISSZA igaz
    VISSZA hamis

```

- + Refuel(amount: int): void: Növeli a zúzalék mennyiségét a megadott értékkel de maximum a rocklimit értékéig.
 - Algoritmus:

```

rockamount = rockamount + amount
HA rockamount > rocklimit:
    rockamount = rocklimit

```

8.1.24 Vehicle (absztrakt)

- Felelősség:

Általános járműveket reprezentáló absztrakt őssztály. Nyilvántartja, hogy a jármű melyik sávon tartózkodik éppen, és kezeli a járművek blokkolódását is.

- Őssztályok:

-

- Interfészek:

-

- Attribútumok:

- currentLane: Lane: A sáv, amelyen a jármű jelenleg tartózkodik.
- blockedTurns: int: A hátralévő körök száma, ameddig a jármű blokkolt.

- Metódusok:

- + Step(): void (absztrakt): A jármű léptetését végző metódus, amelyet a leszármazottak valósítanak meg az egyedi mozgási logikájuk alapján.
- + MeetVehicle(v: Vehicle): void: Két jármű találkozásakor (ütközéskor) lefutó metódus, amely mindkét járművet blokkolja.
- + IsBlocked(): boolean: Visszaadja, hogy a jármű mozgásképtelen-e.
 - Algoritmus:


```

HA blockedTurns > 0:
    VISSZA igaz
    VISSZA hamis

```
- + GetCurrentLane(): Lane: Visszaadja az aktuális sávot.
- + DecrementBlock(): void: Minden körben csökkenti a blokkoltság idejét, ha a jármű blokkolva van.

- + SwitchPassableLane(): boolean: Megkísérli áthelyezni a járművet az út egy szomszédos, járható sávjára, ha a jelenlegi előtt akadály van.
 - Algoritmus:

```
ut = currentLane.GetRoad()
```

```
CIKLUS sav ELEME ut.GetLanes():
```

```
    HA sav != currentLane ÉS sav.IsPassable() == igaz:
```

```
        currentLane = sav
```

```
    VISSZA igaz
```

```
VISSZA hamis
```

8.1.25 Car

- Felelősség:

Személyautót reprezentál, amely önállóan, a legrövidebb úton (PathFinder segítségével) ingázik az otthona és a munkahelye között.

- Ősosztályok:

Vehicle

- Interfészek:

-

- Attribútumok:

- home: Home: Az autó indulási pontja (lakóhelye).
- workplace: Workplace: Az autó célpontja (munkahelye).
- pathFinder: Pathfinder: Az útvonalkeresésért felelős objektum referenciája.

- Metódusok:

- + Step(): void: Végrehajtja az autó mozgását. Kezeli a blokkoltságot és az akadályok kikerülését is.

- Algoritmus

```
    HA IsBlocked() == igaz:
```

```
        DecrementBlock()
```

```
    VISSZA
```

```
    kovetkezoSav = GetNextLane()
```

```
    HA kovetkezoSav != null:
```

```
        HA kovetkezoSav.IsPassable() == igaz:
```

```
            kovetkezoSav.Accept(THIS)
```

```
            currentLane = kovetkezoSav
```

```
    KÜLÖNBEN:
```

```
        HA SwitchPassableLane() == hamis:
```

```
            blockedTurns = 1
```

```

VISSZA
KÜLÖNBEN:
    currentLane.Accept(THIS)

```

```

HA currentLane.SlipperyRoadWithVehicles() == igaz:
    CIKLUS másikJarmu ELEME currentLane.vehicles:
        HA másikJarmu != THIS:
            MeetVehicle(másikJarmu)
        BREAK

```

- + getNextLane(): Lane: Kiszámítja és visszaadja a következő sávot a pathFinder alapján a cél (munkahely vagy otthon) felé.
 - Algoritmus


```

jelenlegiUt = currentLane.GetRoad()
erkezesiCsomopont = jelenlegiUt.to
HA erkezesiCsomopont == home:
    currentTarget = workplace
KÜLÖNBEN HA erkezesiCsomopont == workplace:
    currentTarget = home
utvonalList = pathFinder.GetShortestPath(erkezesiCsomopont,
currentTarget)
HA utvonalList != null ÉS utvonalList NEM ÜRES:
    VISSZA utvonalList[0]
KÜLÖNBEN:
    VISSZA null
                    
```

8.1.26 Bus

- Felelősség:

Egy buszt reprezentál, amely két végállomás között közlekedik. A mozgás irányát alapvetően a buszsofőr (játékos) határozza meg, a busz pedig regisztrálja a teljesített útvonalakat.

- Ősosztályok:

Vehicle

- Interfészek:

-

- Attribútumok:

- terminalStart: Terminal: A busz egyik végállomása.
- terminalEnd: Terminal: A busz másik végállomása.
- completedRounds: int: A sikeresen megtett körök száma.

- Metódusok:

- + Step(): void: Végrehajtja a busz mozgását (a konkrét lépést a játékos bemenete határozza meg)
- + ArrivedAtTerminal(): void: A végállomásra érkezéskor lefutó metódus. Növeli a körök számát és visszavezeti a másik végponthoz.

- Algoritmus:

```
completedRounds = completedRounds + 1
ideiglenes = terminalStart
terminalStart = terminalEnd
terminalEnd = ideiglenes
```

- + GetCompletedRounds(): int: Visszaadja a completedRounds értékét.
- IsRouteComplete(): boolean: Ellenőrzi, hogy a busz befejezte-e az elvárt köröket.

8.1.27 SnowPlow

- Felelősség:
Hókotrót reprezentáló osztály. Képes cserélni a takarítófejét, és ahogy áthalad egy sávon, alkalmazza a rajta lévő fej takarítási logikáját. Tartalmazza az árát és hivatkozik a tulajdonosára.
- Ősosztályok:
-
- Interfészek:
-
- Attribútumok:
 - currentLane: Lane: A sáv, amin a hókotró tartózkodik.
 - head: CleanerHead: A hókotróra jelenleg felszerelt takarítófej.
 - owner: CleanerPlayer: A hókotrót megvásárló és irányító játékos.
 - price: int: A hókotró megvásárlási ára.
- Metódusok:
 - + Step(): void: Végrehajtja a hókotró lépését és meghívja a tisztítást.
 - Algoritmus:


```
HA targetLane != null:
    currentLane = targetLane
    currentLane.Accept(THIS)
    targetLane = null

HA head != null ÉS head.IsOperational() == igaz:
    head.Clean(currentLane)
```
 - + ChangeHead(newHead: CleanerHead): void: Lecseréli az aktuális head referenciát az új fejre.
 - + GetCurrentLane(): Lane: Visszaadja a jelenlegi sávot.
 - + GetHead(): CleanerHead: Visszaadja a rászerezett fejet.
 - + SetTargetLane(lane: Lane): void: Beállítja azt a sávot, ahová a hókotrónak a következő lépésben haladnia kell.
 - + GetOwner(): CleanerPlayer: Visszaadja a tulajdonost.
 - + GetPrice(): int: Visszaadja a hókotró árát.

8.1.28 Player (absztarkt)

- Felelősség:
Absztrakt alapsztály a játékban résztvevő összes játékosra. Egységes interfészt biztosít a körök kezeléséhez és a játékosok azonosításához.
- Ősosztályok:
-
- Interfészek:
-
- Attribútumok:
-name: String: A játékos neve
- Metódusok:
 - + TakeTurn(): void (absztrakt): Ebben a metódusban valósítja meg a játékos a körében végrehajtható akciókat (pl. mozgás, vásárlás).

8.1.29 CleanerPlayer

- Felelősség:
A hókotrókat irányító játékost reprezentálja. Kezeli a saját pénzügyi egyenlegét, új eszközöket és üzemanyagot vásárolhat, valamint koordinálja a tulajdonában lévő hókotró gépeket.
- Ősosztályok:
Player
- Interfészek:
-
- Attribútumok:
 - balance: Int: A játékos aktuális egyenlege.
 - plows: List<SnowPlow>: A játékos irányítása alá tartozó hókotrók listája.
- Metódusok:
 - + TakeTurn(): void: Lehetővé teszi a játékos számára a hókotrók léptetését vagy a vásárlást az adott körben.
 - + BuyHead(plow: SnowPlow, head: CleanerHead): void: Megvásárolja a kiválasztott takarítófejet a megadott hókotróra, ha van rá elég pénze.
 - Algoritmus:


```
ar = head.GetPrice()
HA balance >= ar:
    balance = balance - ar
    plow.ChangeHead(head)
```

KÜLÖNBEN:
VISSZA
 - + BuyFuel(plow: SnowPlow, fuelType: FuelType, amount: Int): void: Üzemanyagot vagy szóróanyagot vásárol a hókotróra szerelt fejhez.
 - Algoritmus:


```
egysegAr = fuelType.GetPrice()
osszkoltseg = egysegAr * amount
HA balance >= osszkoltseg:
    fej = plow.GetHead()
    HA fej != null ÉS fej.Accepts(fuelType):
        balance = balance - osszkoltseg
        fej.Refuel(amount)
```

KÜLÖNBEN:
VISSZA

- + BuyNewPlow(lastPlow: SnowPlow): void: Új hókotró alapgépet vásárol a flottájába.

- Algoritmus:

```
ujKotro = NEW SnowPlow()
```

```
ar = ujKotro.GetPrice()
```

```
HA balance >= ar:
```

```
balance = balance - ar
```

```
plows.ADD(ujKotro)
```

```
HA lastPlow != null:
```

```
celSav = lastPlow.GetCurrentLane()
```

```
ujKotro.SetTargetLane(celSav)
```

```
celSav.Accept(ujKotro)
```

KÜLÖNBEN:

VISSZA

- + ReceivePayment(amount: Int): void: A játékos jutalmat kap a sikeres takarítás után.
- + GetBalance(): Int: Visszaadja a játékos aktuális egyenlegét.

8.1.30 BusDriver

- Felelősség:
A buszt irányító sofőr reprezentációja. Feladata a rábízott busz mozgatása a menetrend szerinti útvonalon a végállomások között.
- Ősosztályok:
Player
- Interfészek:
-
- Attribútumok:
- bus: Bus: Az irányított busz referenciája.
- Metódusok:
 - + TakeTurn(): void: A sofőr a körében eldönti, hogy merre lépteti a buszt a szomszédos sávok közül.

8.1.31 FuelType

- Felelősség:
Absztrakt alaposztály a különböző üzemanyagok és szóróanyagok egységes kezelésére.
- Ősosztályok:
-
- Interfészek:
-

- Attribútumok:
 - price: int: Az anyag egységára.
- Metódusok:
 - + GetPrice(): int: Visszaadja a vásárláskori árat.

8.1.32 Rock

- Felelősség:
- Ősosztályok:
 - FuelType
- Interfészek:
 -
- Attribútumok:
 - price: int: Az anyag egységára.
- Metódusok:
 - + GetPrice(): int: Visszaadja a vásárláskori árat.

8.1.33 Salt

- Felelősség:
 - Só típusú szóróanyag reprezentálása. Ezt a fogyóeszközt a SaltHead használja.
- Ősosztályok:
 - FuelType
- Interfészek:
 -
- Attribútumok:
 - price: int: Az anyag egységára.
- Metódusok:
 - + GetPrice(): int: Visszaadja a vásárláskori árat.

8.1.34 Kerosene

- Felelősség:
 - Kerozin típusú üzemanyag reprezentálása. Ezt az üzemanyagot a DragonHead használja.
- Ősosztályok:
 - FuelType
- Interfészek:
 -

- Attribútumok:
 - price: int: Az anyag egységára.
- Metódusok:
 - + GetPrice(): int: Visszaadja a vásárláskori árat.

8.2 A tesztek részletes tervei, leírásuk a teszt nyelvén

8.2.1 Teszteset1 (T01 - Autó léptetése)

- **Leírás**

Egy autó van a pályán egy adott célállomással. Az autót léptetjük az útvonalkereső által megadott irányba. A teszt az autó új pozícióját vizsgálja a kimeneten.

- **Ellenőrzött funkcionalitás, várható hibahelyek**

Az autó normál haladásának és útvonalkövetésének ellenőrzése. Hibahely lehet az útvonalkereső elszállása, vagy ha az autó Blocked=true állapotba kerül.

- **Bemenet**

Csomopont c1 Home
 Csomopont c2 Workplace
 Ut u1
 Sav s1
 Hozzaad_Sav u1 s1
 Kapcsol u1 c1 c2
 Jarmu auto1 Car
 Uticel_Auto auto1 c1 c2
 Raallit auto1 s1
 Auto_Lep auto1 false
 Stat Car auto1

- **Elvárt kimenet**

grep -q "auto1.*Sikeres lepes" -> Jelzi az akció sikerességét.
 grep -q "Car.*auto1.*Position=s1" -> Jó sávon áll.
 grep -q "Blocked=false" -> Nincs elakadva.

8.2.2 Teszteset2 (T02 - Jármű megcsúszása és ütközése)

- **Leírás**

Egy jeges sávon van egy busz. Egy személyautót léptetünk ugyan erre a sávra, ami megcsúszik és nekiütközik a busznak. Mindkét jármű elakad.

- **Ellenőrzött funkcionalitás, várható hibahelyek**

A járművek jeges úton való megcsúszásának és ütközésének ellenőrzése.

- **Bemenet**

Csomopont c1 Home
 Csomopont c2 Workplace
 Ut u1
 Sav s1
 Hozzaad_Sav u1 s1
 Kapcsol u1 c1 c2
 Allapot s1 IcyState
 Jarmu busz1 Bus

```
Jarmu auto1 Car
Raallit busz1 s1
Uticel_Auto auto1 c1 c2
Raallit auto1 s1
Auto_Lep auto1 false
Stat Car auto1
Stat Bus busz1
```

- **Elvárt kimenet**

```
grep -q "auto1.*Utkozes" vagy grep -q "megcsuszott" -> Az ütközés eseményének logja.
grep -q "Car.*auto1.*Blocked=true" -> Az autó blokkolódott.
grep -q "Bus.*busz1.*Blocked=true" -> A busz is blokkolódott.
```

8.2.3 Teszteset3 (T03 - Hókotró takarít - Hányó fejjel)

- **Leírás**

Egy hányó fejjel felszerelt hókotró áll a vastag hóval borított sáv előtt. A hókotró áthalad a sávon. A sáv tiszta állapotba kerül.

- **Ellenőrzött funkcionalitás, várható hibahelyek**

A hányó fejjel történő hóeltakarítás működésének ellenőrzése, valamint a Takarító játékos összekötése a hókotróval.

- **Bemenet**

```
Ut u1
Sav s1
Hozzaad_Sav u1 s1
Allapot s1 ThickSnowState
Hokotro hk1
Felszerel hk1 Throw
Takarito hk1 0
Raallit_h hk1 s1
Hokotro_Lep hk1 u1 s1
Stat Lane s1
```

- **Elvárt kimenet**

```
grep -q "hk1: .* takarított" -> Az akció lefutott.
grep -q "s1.*allapota clearstatebe változott" -> Új, kért kimeneti forma.
grep -q "Takarito.*penzt szerzett" -> Pénzszerzés ellenőrzése.
grep -q "Lane.*s1.*SnowThickness=0" -> Nem maradt rajta hó.
```

8.2.4 Teszteset4 (T04 - Zúzalék szórása jeges úton)

- **Leírás**

Egy zúzalékszóró fejjel felszerelt hókotró áthalad egy jeges sávon. Ezt követően egy autót léptetünk ugyanerre a sávra, ami a kiszórt zúzalék miatt nem csúszik meg.

- **Ellenőrzött funkcionalitás, várható hibahelyek**

A zúzalék kiszóródása és hatásának ellenőrzése. Hibahely lehet, ha a sáv állapota nem veszi figyelembe a zúzalékot, és a jármű mégis megcsúszik.

- **Bemenet**

```
Ut u1
Sav s1
Hozzaad_Sav u1 s1
Allapot s1 IcyState
Hokotro hk1
Felszerel hk1 Rock
```

Takarito hk1 0
 Toltes hk1 Rock 5
 Raallit_h hk1 s1
 Hokotro_Lep hk1 u1 s1
 Jarmu auto1 Car
 Raallit auto1 s1
 Auto_Lep auto1 false
 Stat Lane s1
 Stat SnowPlow hk1

- **Elvárt kimenet**

grep -q "hk1.*Zuzalek szorasa sikeres" -> A hókotró kiszórta a követ.
 grep -q "Lane.*s1.*State=IcyState.*Slippery=false" -> A sáv jeges maradt, de nem csúszik.
 grep -q "SnowPlow.*hk1.*Fuel_Rock=4" -> A zúzalékkészlet csökkent.

8.2.5 Teszteset5 (T05 - Havazás és jégképződés letaposás miatt)

- **Leírás**

Egy vékony havas sávon autók haladnak át ismétlődően. Az áthaladási küszöb elérésekor a hó jéggé tömörödik.

- **Ellenőrzött funkcionalitás, várható hibahelyek**

Sikeresen alakul-e jéggé a letaposott hó. Hibahely lehet a számláló helytelen működése vagy az állapotváltás elmaradása.

- **Bemenet**

Ut u1
 Sav s1
 Hozzaad_Sav u1 s1
 Allapot s1 ThinSnowState
 Jarmu auto1 Car
 Raallit auto1 s1
 Auto_Lep auto1 false
 Auto_Lep auto1 false
 Auto_Lep auto1 false
 Stat Lane s1

- **Elvárt kimenet**

grep -q "s1.*allapota IcyState-re változott" -> Jelzi az eseményt a logban.
 grep -q "Lane.*s1.*State=IcyState" -> A stat parancs is a jég állapotot mutatja.

8.2.6 Teszteset6 (T06 - Busz haladása két végállomás között)

- **Leírás**

Egy busz úton van a megadott végállomása felé a buszsofőr irányításával. A busz megérkezik a céljához.

- **Ellenőrzött funkcionalitás, várható hibahelyek**

A busz végállomások közötti közlekedésének ellenőrzése és a Buszsofor játékos inicializálása.

- **Bemenet**

Csomopont v1 Terminal
 Csomopont v2 Terminal
 Ut u1
 Sav s1
 Hozzaad_Sav u1 s1
 Kapcsol u1 v1 v2

Jarmu busz1 Bus
 Buszsofor busz1
 Uticel_Busz busz1 v1 v2
 Raallit busz1 s1
 Busz_Lep busz1 u1 s1 false
 Stat Bus busz1

- **Elvárt kimenet**

grep -q "busz1.*Sikeres lepes" -> Jelzi, hogy a busz lépett.
 grep -q "Bus.*busz1.*Position=" -> A jármű statisztikája lekérdezhető.

8.2.7 Teszteset7 (T07 - Hókotró takarít - Söprő fejjel)

- **Leírás**

Egy söprő fejjel felszerelt hókotró áthalad egy vékony havon. A gép a havat a szomszédos sávba tolja át, ami megnöveli az ottani hóvastagságot.

- **Ellenőrzött funkcionalitás, várható hibahelyek**

Megfelelően tolódik-e át a szomszédos sávra a hó. Hibahely lehet, ha az aktuális sáv nem tisztul le, vagy a szomszéd sávon nem nő a hó vastagsága.

- **Bemenet**

Ut u1
 Sav s1
 Sav s2
 Hozzaad_Sav u1 s1
 Hozzaad_Sav u1 s2
 Allapot s1 ThinSnowState
 Allapot s2 ClearState
 Hokotro hk1
 Felszerel hk1 Sweep
 Takarito hk1 0
 Raallit_h hk1 s1
 Hokotro_Lep hk1 u1 s1
 Stat Lane s1
 Stat Lane s2

- **Elvárt kimenet**

grep -q "Lane.*s1.*State=ClearState" -> A takarított sáv megtisztult.
 grep -q "Lane.*s2.*State=ThinSnowState" vagy grep -q "Lane.*s2.*SnowThickness=" -> A szomszéd sávra átkerült a hó.

8.2.8 Teszteset8 (T08 - Hókotró takarít - Jégtörő fejjel)

- **Leírás**

Egy jégtörő fejjel felszerelt hókotró áthalad egy vastag jéggel borított sávon. A sávon lévő jég feltörik.

- **Ellenőrzött funkcionalitás, várható hibahelyek**

Sikeresen töri-e fel a jégtörő a jeget. Hibahely, ha a sáv IcyState marad BrokenIceState helyett.

- **Bemenet**

Ut u1
 Sav s1
 Hozzaad_Sav u1 s1
 Allapot s1 IcyState
 Hokotro hk1

Felszerel hk1 IceBreaker
 Takarito hk1 0
 Raallit_h hk1 s1
 Hokotro_Lep hk1 u1 s1
 Stat Lane s1

- **Elvárt kimenet**

grep -q "s1 allapota BrokenIceState-re változott" -> Log üzenet a jég feltöréséről.
 grep -q "Lane.*s1.*State=BrokenIceState" -> A stat is a feltört jeget mutatja.

8.2.9 Teszteset9 (T09 - Hókotró takarít - Sósóró fejjel)

- **Leírás**

Egy sósóró fejjel felszerelt hókotró halad át egy jeges sávon. Az olvadási folyamat megindul, a gép sókészlete csökken.

- **Ellenőrzött funkcionalitás, várható hibahelyek**

A sósóró fejjel történő takarítás és a sófogyás ellenőrzése.

- **Bemenet**

Ut u1
 Sav s1
 Hozzaad_Sav u1 s1
 Allapot s1 IcyState
 Hokotro hk1
 Felszerel hk1 Salt
 Takarito hk1 0
 Toltes hk1 Salt 5
 Raallit_h hk1 s1
 Hokotro_Lep hk1 u1 s1
 Stat SnowPlow hk1

- **Elvárt kimenet**

grep -q "s1.*sozva.*olvadási folyamat elindult" -> Sikeres sózás üzenete.
 grep -q "SnowPlow.*hk1.*Fuel_Salt=4" vagy grep -q "Maradék so: 4" -> A sókészlet csökkentését ellenőrzi.

8.2.10 Teszteset10 (T10 - Hókotró takarít - Sárkány fejjel)

- **Leírás**

Egy sárkányfejjel felszerelt hókotró áthalad egy vastag jéggel borított sávon. A sáv azonnal megtisztul, a hókotró kerozinkészlete csökken.

- **Ellenőrzött funkcionalitás, várható hibahelyek**

A sárkányfejjel történő úttisztítás és a kerozinfogyás ellenőrzése.

- **Bemenet**

Ut u1
 Sav s1
 Hozzaad_Sav u1 s1
 Allapot s1 IcyState
 Hokotro hk1
 Felszerel hk1 Dragon
 Takarito hk1 0
 Toltes hk1 Kerosene 10
 Raallit_h hk1 s1
 Hokotro_Lep hk1 u1 s1
 Stat Lane s1

Stat SnowPlow hk1

- **Elvárt kimenet**

grep -q "Lane.*s1.*State=ClearState" -> A sáv azonnal tiszta lett az olvasztástól.

grep -q "SnowPlow.*hk1.*Fuel_Kerosene=9" vagy grep -q "Maradek Kerosene: 9" -> A kerozin fogyása.

8.2.11 Teszteset11 (T11 - Autó kikerüli a vastag havat)

- **Leírás**

Egy autó halad az úton, de a következő sávot vastag hó borítja. Az autó automatikusan áttér a tiszta szomszédos sávra az elakadás helyett.

- **Ellenőrzött funkcionalitás, várható hibahelyek**

Megfelelően cselekszik-e az autó szükséges sávváltás esetén. Hibahely, ha az autó belerohan a hóba és elakad a szomszédos sáv használata helyett.

- **Bemenet**

Ut u1

Sav s1

Sav s2

Hozzaad_Sav u1 s1

Hozzaad_Sav u1 s2

Allapot s1 ThickSnowState

Allapot s2 ClearState

Jarmu auto1 Car

Raallit auto1 s1

Auto_Lep auto1 false

Stat Car auto1

- **Elvárt kimenet**

grep -q "auto1.*kikerulte az akadalyt.*uj pozicio: s2" -> Log üzenet a kikerülésről.

grep -q "Car.*auto1.*Position=s2" -> A pozíció ténylegesen s2 lett.

8.2.12 Teszteset12 (T12 - Sózott sávra eső hó)

- **Leírás**

Egy korábban besózott sávra havazik. A lerakódó hó az aktív só miatt azonnal elolvad.

- **Ellenőrzött funkcionalitás, várható hibahelyek**

Megmarad-e a hó egy lesózott sávon. Hibahely, ha az állapot átvált havasra, mert a só hatása elveszett.

- **Bemenet**

Ut u1

Sav s1

Hozzaad_Sav u1 s1

Allapot s1 ClearState

Hokotro hk1

Felszerel hk1 Salt

Takarito hk1 0

Toltes hk1 Salt 5

Raallit_h hk1 s1

Hokotro_Lep hk1 u1 s1

Ho s1 3

Stat Lane s1

- **Elvárt kimenet**

grep -q "s1.*ho esett, de a so azonnal felolvasztotta" -> A havazás és az olvadás nyugtázása.
 grep -q "Lane.*s1.*State=ClearState" -> A sáv a havazás ellenére tiszta marad.
 grep -q "Lane.*s1.*SnowThickness=0" -> Hóvastagság továbbra is 0 marad.

8.2.13 Teszteset13 (T13 - Kavics áttolása szomszédos sávra söprő fejjel)

- **Leírás**

Egy söprő (Sweep) fejjel felszerelt hókotró áthalad egy olyan sávon, amire előzetesen kavicsot szórtunk. A gép a kavicsot át kell, hogy tolja a szomszédos sávra. A takarító játékos az akcióért pénzt kap.

- **Ellenőrzött funkcionalitás, várható hibahelyek**

A SweepHead működése kavics (zúzalék) esetén, illetve a Takarító játékos pénzjutalmának jóváírása. Hibahely lehet, ha a kavics eltűnik, vagy ha az aktuális sávon marad áttolás helyett.

- **Bemenet**

Ut u1
 Sav s1
 Sav s2
 Hozzaad_Sav u1 s1
 Hozzaad_Sav u1 s2
 Kavics s1
 Hokotro hk1
 Felszerel hk1 Sweep
 Takarito hk1 0
 Raallit_h hk1 s1
 Hokotro_Lep hk1 u1 s1
 Stat Lane s1
 Stat Lane s2

- **Elvárt kimenet**

grep -q "hk1: .* takarított" -> A takarítás akció sikeres lefutása.
 grep -q "s1.*allapota clearstatebe változott" -> A takarított sáv állapota frissült.
 grep -q "Takarito.*penzt szerzett" -> A játékos megkapta a fizetését.
 grep -q "Lane.*s1.*Rocky=false" -> A kavics eltűnt az eredeti sávról.
 grep -q "Lane.*s2.*Rocky=true" -> A kavics megjelent a szomszédos sávon.

8.2.14 Teszteset14 (T14 - Sikertelen vásárlás / Nincs elég pénz)

- **Leírás**

A takarító játékos megpróbál venni egy Sárkányfejet és kerozint a hókotrójába, de a kezdeti egyenlege 0. Mivel nincs pénze, a vásárlás sikertelen lesz, így amikor a gép egy jeges sávra lép, nem tudja elolvasztani a jeget.

- **Ellenőrzött funkcionalitás, várható hibahelyek**

A CleanerPlayer egyenlegkezelésének és a vásárlás (illetve üzemanyagfeltöltés) megtagadásának ellenőrzése. Hibahely lehet, ha a játékos 0 pénzből is tud tölteni, és a sáv mégis megtisztul.

- **Bemenet**

Ut u1
 Sav s1
 Hozzaad_Sav u1 s1
 Allapot s1 IcyState
 Hokotro hk1
 Felszerel hk1 Dragon

Takarito hk1 0
 Toltes hk1 Kerosene 10
 Raallit_h hk1 s1
 Hokotro_Lep hk1 u1 s1
 Stat Lane s1
 Stat SnowPlow hk1

- **Elvárt kimenet**

grep -q "sikertelen vasarlas" vagy grep -q "[ERROR\].*Nincs elegendo" -> A rendszer hibát vagy sikertelen akciót jelez a töltésnél.

grep -q "Lane.*s1.*State=IcyState" -> A jég ott maradt, mivel nem volt üzemanyag az olvasztáshoz.

grep -q "SnowPlow.*hk1.*Fuel_Kerosene=0" -> A kerozinszint valóban 0 maradt.

8.2.15 Teszteset15 (T15 - Kifogyott üzemanyaggal való takarítási kísérlet)

- **Leírás**

Egy hókotró rálép egy jeges sávra felszerelt Sósózóró fejjel, de a sókészlete már teljesen üres (0). A teszt ellenőrzi, hogy a jég tényleg ott marad-e, és nem történik olvasztás (nincs nullosztás vagy negatívba menő készlet).

- **Ellenőrzött funkcionalitás, várható hibahelyek**

A SaltHead IsOperational() metódusának ellenőrzése, amelynek üres készlet esetén hamisat kell visszaadnia, blokkolva a takarítást.

- **Bemenet**

Ut u1
 Sav s1
 Hozzaad_Sav u1 s1
 Allapot s1 IcyState
 Hokotro hk1
 Felszerel hk1 Salt
 Takarito hk1 100
 Toltes hk1 Salt 0
 Raallit_h hk1 s1
 Hokotro_Lep hk1 u1 s1
 Stat Lane s1
 Stat SnowPlow hk1

- **Elvárt kimenet**

grep -q "Lane.*s1.*State=IcyState" -> Az állapot IcyState maradt, nem indult meg az olvadás.

grep -q "SnowPlow.*hk1.*Fuel_Salt=0" -> A sókészlet 0 maradt, nem ment negatívba.

Biztosítani kell, hogy a kimenet NE tartalmazza az "olvadási folyamat elindult" szöveget (ezt a szkriptben pl. grep -v-vel lehetne ellenőrizni, de a stat ellenőrzés is elég).

8.2.16 Teszteset16 (T16 - Útvonalkereső "Nincs út" esete baleset miatt)

- **Leírás**

Két egymást követő útszakaszból a másodikon egy busz és egy autó balesetet szenved, amivel teljesen blokkolják a sávot. Egy második autó próbál eljutni a munkahelyre ezen az útvonalon, de a PathFinder nem talál szabad utat.

- **Ellenőrzött funkcionalitás, várható hibahelyek**

A Pathfinder és a gráf bejárásának helyes működése olyankor, amikor egyetlen lehetséges útvonal sincs (mert minden sáv Blocked). Hibahely lehet a program végtelen ciklusba kerülése vagy NullPointerException dobása.

- **Bemenet**

```
Csomopont c1 Home
Csomopont c2 Intersection
Csomopont c3 Workplace
Ut u1
Sav s1
Hozzaad_Sav u1 s1
Kapcsol u1 c1 c2
Ut u2
Sav s2
Hozzaad_Sav u2 s2
Kapcsol u2 c2 c3
Allapot s2 IcyState
Jarmu busz1 Bus
Jarmu auto1 Car
Raallit busz1 s2
Uticel_Auto auto1 c2 c3
Raallit auto1 s2
Auto_Lep auto1 false
Jarmu auto2 Car
Uticel_Auto auto2 c1 c3
Raallit auto2 s1
Auto_Lep auto2 false
Stat Car auto2
```

- **Elvárt kimenet**

grep -q "[ERROR\].*Nincs ut" vagy grep -q "nem tud lepni" -> A rendszer logolja, hogy az auto2 számára nincs járható útvonal a baleset (blokkolt s2 sáv) miatt.

grep -q "Car.*auto2.*Position=s1" -> Az auto2 nem lépett le az eredeti sávjáról, mivel nem talált utat.

8.2.17 Teszteset17 (T17 - Havazás vékony havas sávra - vastagodás)

- **Leírás**

Egy már vékony hóval borított sávra újabb hó esik. A teszt ellenőrzi, hogy a hóvastagság növekedésével a sáv állapota átvált-e vastag havas (ThickSnowState) állapotba, ami már elakasztja a járműveket.

- **Ellenőrzött funkcionalitás, várható hibahelyek**

Az állapotváltás logikájának ellenőrzése a ThinSnowState osztályban. Hibahely, ha a hóvastagság nő, de a sáv járható marad az autók számára.

- **Bemenet**

```
Ut u1
Sav s1
Hozzaad_Sav u1 s1
Allapot s1 ThinSnowState
Ho s1 4
Stat Lane s1
```

- **Elvárt kimenet**

grep -q "Lane.*s1.*State=ThickSnowState" -> A sáv sikeresen átváltott vastag havas állapotba.

grep -q "Lane.*s1.*IsPassable=false" -> A sáv már nem járható a normál járműveknek.

8.2.18 Teszteset18 (T18 - Új hókotró vásárlás)

- **Leírás**

A takarító játékos elegendő pénzzel rendelkezik, és vásárol egy új hókotrót. A gép megjelenik a pályán, a játékos egyenlege pedig a gép árával csökken.

- **Ellenőrzött funkcionalitás, várható hibahelyek**

A CleanerPlayer vásárlási logikájának ellenőrzése. Hibahely, ha a gép megjelenik, de a pénz nem vonódik le, vagy ha rossz sávon jön létre.

- **Bemenet**

Ut u1

Sav s1

Hozzaad_Sav u1 s1

Hokotro hk1

Raallit_h hk1 s1

Takarito cp1 1000

Vesz_Hokotro cp1 hk1

Stat CleanerPlayer cp1

Stat SnowPlow hk2

- **Elvárt kimenet**

grep -q "cp1.*vasarlas sikeres" -> A tranzakció sikeres volt.

grep -q "CleanerPlayer.*cp1.*Balance=500" -> Az egyenleg pontosan a hókotró árával csökkent.

grep -q "SnowPlow.*hk2.*Position=s1" -> Az új gép sikeresen létrejött ugyanazon a sávon, ahol az eredeti gép is állt.

8.2.19 Teszteset19 (T19 - Fejcsere)

- **Leírás**

A takarító játékos lecseréli az egyik hókotrójának fejét egy újra (pl. Söprőről Hányóra). A teszt ellenőrzi az egyenleg csökkenését és az új fej felszerelését.

- **Ellenőrzött funkcionalitás, várható hibahelyek**

A ChangeHead metódus és a vásárlási tranzakció helyes működése. Hibahely, ha a régi fej megmarad, vagy ha a pénzlevonás elmarad.

- **Bemenet**

Hokotro hk1

Felszerel hk1 Sweep

Takarito cp1 500

Vesz_Fej cp1 hk1 Throw

Stat SnowPlow hk1

Stat CleanerPlayer cp1

- **Elvárt kimenet**

grep -q "CleanerPlayer.*cp1.*Balance=300" -> A fej ára levonódott.

grep -q "SnowPlow.*hk1.*Head=ThrowHead" -> Az új fej sikeresen felszerelésre került.

8.2.20 Teszteset20 (T20 - Hókotró takarít - sáv már tiszta)

- **Leírás**

Egy hókotró rálép egy teljesen tiszta (ClearState) sávra. A gép nem végez érdemi takarítást, és a takarító játékos nem kap érte pénzt.

- **Ellenőrzött funkcionalitás, várható hibahelyek**

A pontszerzési exploitok elkerülésének ellenőrzése. Hibahely, ha a ClearState sáv "letisztításáért" is pénzt oszt a rendszer.

- **Bemenet**

```
Ut u1
Sav s1
Hozzaad_Sav u1 s1
Allapot s1 ClearState
Hokotro hk1
Felszerel hk1 Throw
Takarito cp1 0
Raallit_h hk1 s1
Hokotro_Lep hk1 u1 s1
Stat CleanerPlayer cp1
```

- **Elvárt kimenet**

grep -q "CleanerPlayer.*cp1.*Balance=0" -> A játékos egyenlege 0 maradt, nem kapott ingyen pénzt.

8.2.21 Teszteset21 (T21 - Jármű blokkolt - nem lép)

- **Leírás**

Egy jármű, amely korábban balesetet szenvedett, Blocked=2 állapotban van. A tesztelő kiad egy léptetési parancsot, de a gép nem mozdul, csak a blokkolt körök száma csökken 1-re.

- **Ellenőrzött funkcionalitás, várható hibahelyek**

A DecrementBlock metódus és az IsBlocked állapot ellenőrzése. Hibahely, ha a jármű blokkoltan is képes sávot váltani.

- **Bemenet**

```
Ut u1
Sav s1
Hozzaad_Sav u1 s1
Jarmu auto1 Car
Raallit auto1 s1
Allapot_Auto auto1 Blocked=2
Auto_Lep auto1 false
Stat Car auto1
```

- **Elvárt kimenet**

```
grep -q "auto1.*blokkolt, nem tud lepni" -> Log üzenet a blokkolás érvényesüléséről.
grep -q "Car.*auto1.*BlockedTurns=1" -> A visszaszámláló pontosan 1-gyel csökkent.
grep -q "Car.*auto1.*Position=s1" -> A jármű nem mozgult el eredeti helyéről.
```

8.3 A tesztelést támogató programok tervei

A tesztelési keretrendszert operációs rendszer szintű parancsfájlokkal (Bash Linux/macOS-en) valósítjuk meg, ami az alábbi fő részből áll:

1. Tesztadatok előállítása: A tesztbemeneteket az előzőleg specifikált Use-Case-ek alapján, manuálisan hozzuk létre .txt kiterjesztésű fájlokként (pl. tesztek/1.txt). Ezek a fájlok pontosan a prototípus bemeneti nyelvén írják le a végrehajtandó lépéseket.

2. Fordító szkript: Lefuttatásakor a szkript megkeresi a projektben található összes .java forrásfájlt, és a Java fordító (javac) segítségével lefordítja őket, készen állva a tesztelésre.

3. Futtató és kiértékelő szkript: A konzulensi iránymutatásoknak megfelelően a merev, teljes szövegegyezést elváró (pl. diff alapú) ellenőrzés helyett rugalmas, kulcsszavas keresést alkalmazunk, mivel a kimenet minimális formázási eltérése is fals hibát dobna.

A szkript működése a következő:

- A cat paranccsal beolvassa a bemeneti fájlt és csővezetéken (|) átadja a Java prototípusnak.
- A kimenetet egy .out fájlba irányítja (pl. 1.out).
- A kiértékeléshez a grep eszközzel rákeres a várt események (pl. állapotváltás, sikeres lépés) kulcsszavaira, majd a wc -l paranccsal megszámlolja a találatokat. Ha a kapott sorok száma megfelelő, a teszt sikeres.

Példa a kiértékelő szkript logikájára (a konzulensi egyeztetés alapján):

Bash

Prototípus futtatása a bemenettel

```
cat tesztek/1.txt | java Proto > 1.out
```

Elvárt események ellenőrzése grep és wc -l kombinációjával

```
SIKERES_LEPES=$(cat 1.out | grep "auto1.*Sikeres lepes" | wc -l)
```

```
JO_POZICIO=$(cat 1.out | grep "Car.*auto1.*Position=s1" | wc -l)
```

```
if [ "$SIKERES_LEPES" -ge 1 ] && [ "$JO_POZICIO" -ge 1 ]; then
```

```
    echo "[ SIKERES ] T01 - Autó léptetése"
```

```
else
```

```
    echo "[ HIBA ] T01 - Autó léptetése nem sikerült!"
```

```
fi
```

8.4 Napló

Kezdet	Időtartam	Résztevők	Leírás
2026.04.20 19:00	1 óra	Tullner Stumpf Maruzsán	Értekezlet.
2026.04.20 20:00	2 óra	Tullner	Tesztesetek kidolgozása
2026.04.20 20:00	2 óra	Stumpf	Osztályok és metódusok leírása
2026.04.21 00:01	2 óra	Dekkert	Osztályok és metódusok leírása
2026.04.21 10:00	2 óra	Dekkert, Kiss, Stumpf	Dokumentum javítása, formázása